

Overview:

This project provided the opportunity to develop a BoggleSolver class to find all valid words that could be created by traversing a Boggle board using a dictionary. To create a solution to this problem, I first developed a trie for the dictionary that enabled fast prefix checking. Then I utilized a depth-first search (DFS) to traverse all possible paths for creating words on the Boggle board. The DFS tracked visited cells on the board, to help enforce the rules of Boggle, and prevent reuse of a cell in the same word. A set was also utilized to collect all found words to avoid listing duplicates. Finally, the list of words was sorted prior to returning them, to provide consistent order of output. Therefore, the overall design of the BoggleSolver struck a balance of being correct, efficient, and easy to understand and utilize the trie to eliminate search paths, thereby limiting the amount of unnecessary DFS searches.

Feedback I Received:

My partner suggested that I implement logic into the program that would automatically add a "U" onto the board when the letter "Q" appeared, since in standard Boggle game rules, "QU" are considered one tile. My partner also suggested optimizing the DFS to check that the row and column indices are inside bounds, prior to calling each other recursively, to limit the number of times the function would call itself unnecessarily, thus slightly improving performance. Both recommendations aimed at improving the overall quality of the implementation and to do so in a manner that maintained correctness.

Feedback I Gave:

I noticed that the use of the "#" character to indicate the end of a word within the trie is a clean and very common method. However, defining "#" as a class-level constant, will improve the readability and maintainability of the code. I also observed that although the empty grids were well-handled, empty rows (i.e. [[]]), were not being checked, therefore adding the checks if not grid and/or if not grid[0], will make the implementation more robust. In general, the code demonstrated a solid grasp of the concepts of recursion, DFS, and trie data structures, effectively eliminating search paths and enforcing the Boggle rules, while handling common edge cases, and efficiently removing duplicates from the word list.

Improvements I Implemented:

I made a number of changes to increase both the robustness and performance of the code. I implemented bounds checking within the DFS loops, to limit the number of times the function will call itself, and thus limit the number of unnecessary recursive calls. I also implemented logic to automatically add a "U", onto the board, after a "Q" has been placed, to conform to the rules of Boggle. I defined a class-level constant called END_SYMBOL to define the end-of-word

marker in the trie, for better readability and maintainability. I also modified the grid validation to include checks for empty rows, to ensure that the solver does not fail due to edge cases. These changes increased both the correctness and clarity of the code.

Static Analysis Results:

To address linting and style issues, I made several changes to improve the readability and maintainability of the code. These changes included correcting inconsistent indentation and spacing, renaming variables in the DFS for clarity, and adding comments to describe the key components of the trie and the DFS. These changes made the code more readable and compliant with the best practices of Python, without changing the basic logic or operation of the code.

Regression Testing:

Following the implementation of the changes, I performed regression testing to verify the correctness of the code. I ran the solver on a variety of test boards, including those with edge cases (empty grids, single 'Q' tiles, etc.), and validated that the output from the solver accurately reflected the expected word lists, with no duplicates. I also tested the performance of the solver with larger dictionaries to confirm that the optimizations to the DFS and trie were effective. The results of these tests confirmed that the modifications did not introduce any new bugs, and that they had improved the performance where the original code lacked optimization.

Reflection:

Through this experience, I learned the value of providing and receiving constructive and specific feedback in the process of reviewing code. Through reviewing my partner's code, I identified ways in which the code could be improved for maintainability, and through receiving feedback, I became more aware of how to optimize performance, and how to apply common standards and best practices (such as handling 'QU'). The experience reinforced many aspects of professional workflow, including documentation of code, defensive programming, testing for correctness, and continuous iteration towards improvement. The process also showed the benefit of collaboration in the code review process, resulting in improved quality of the code and knowledge of the developers involved.